

Strings and Bit Manipulation

Strings as Data Structures

In most languages including Python, strings are immutable sequences of characters. Because they're immutable, operations like concatenation in a loop can be $O(n^2)$ overall if done carelessly — use a list and `".join()"` instead for building strings incrementally, which is $O(n)$.

Common String Techniques

- Two Pointers: one pointer from the start, one from the end, moving toward each other — used for palindrome checks, reversing.
- Sliding Window: maintain a window of characters and expand/shrink it — used for longest substring problems.
- Hash Map of character frequency — used for anagram checks, first unique character.

String Code Patterns

```
# Efficient string building -  $O(n)$ 
```

```
parts = []
```

```
for word in ["hello", "world"]:
```

```
    parts.append(word)
```

```
result = " ".join(parts)
```

```
# Palindrome check - two pointers,  $O(n)$ 
```

```
def is_palindrome(s):
```

```
    left, right = 0, len(s) - 1
```

```
    while left < right:
```

```
        if s[left] != s[right]:
```

```
            return False
```

```
        left += 1
```

```
        right -= 1
```

```
    return True
```

```
# Sliding window - longest substring without repeating chars
```

```
def longest_unique_substring(s):
```

```

seen = {}

left = max_len = 0

for right, char in enumerate(s):
    if char in seen and seen[char] >= left:
        left = seen[char] + 1
    seen[char] = right
    max_len = max(max_len, right - left + 1)

return max_len

```

Bit Manipulation Basics

Bit manipulation operates directly on the binary representation of numbers. It's used for extremely fast, memory-efficient operations and shows up in a specific, recognizable category of interview problems.

Key Bitwise Operators

- AND (&): 1 only if both bits are 1. Used to check/clear specific bits.
- OR (|): 1 if either bit is 1. Used to set specific bits.
- XOR (^): 1 if bits differ. Used to find unique elements, swap without a temp variable.
- NOT (~): flips all bits.
- Left Shift (<<): shifts bits left, equivalent to multiplying by 2 per shift.
- Right Shift (>>): shifts bits right, equivalent to dividing by 2 per shift.

Bit Tricks in Python

```

# Check if a number is even/odd

n = 7

is_odd = n & 1 # 1 if odd, 0 if even


# Find the single non-repeating number where every other appears twice

def single_number(nums):
    result = 0

    for num in nums:
        result ^= num # XOR cancels out pairs

    return result

```

```
# Count set bits (1s) in a number
```

```
def count_set_bits(n):
```

```
    count = 0
```

```
    while n:
```

```
        count += n & 1
```

```
        n >>= 1
```

```
    return count
```

Common Interview Problems

- Longest Palindromic Substring.
- Valid Anagram check using character frequency.
- Single Number — find the element that appears once using XOR.
- Count set bits in a number (Brian Kernighan's Algorithm).
- Check if a number is a power of two using bit tricks.
- Longest substring without repeating characters (sliding window).